BrainBlock DRL Project

Design Choices & Task Plan

Group Plan

May 17, 2026

1 Design Decisions (with Pros & Cons)

1.1 Invalid Action Handling

When the agent selects an action that results in an illegal placement (out-of-bounds or overlap):

Option	Description	Pros	Cons
A) Hard term	Episode ends immediately on any invalid action	Simple to implement; learns to avoid invalid quickly	Few steps early in training → slow learning
B) Neg reward + skip	Give negative reward, skip turn, keep piece	Keeps playing → more learning signal; forgiving	Agent may "farm" negative rewards; longer episodes
C) Action masking	Mask out invalid actions	No wasted steps; fastest learning	Complex implementation; must compute mask
D) Neg reward + retry	Agent retries until valid	Good exploration	Can get stuck in loops; adds hyperparameter

Recommendation: Start with **C (action masking)** for the main experiments. It is the cleanest and most efficient. Optionally compare with **A** as a second reward-function experiment.

1.2 State Representation

Option	Description	Pros	Cons
A) Multi-channel binary	Ch0: board. Ch1–5: one-hot piece. Ch6+: remaining	Spatial structure preserved; works well with CNNs	Small board (8x5) means CNN may be overkill
B) Flat vector	Flatten everything to 50-dim vector	Simple; works well with MLPs; easy to debug	Loses spatial relationships between cells
C) Grid + inventory	Board as channel, shape rendered, counts broadcast	Richest spatial info; strong CNN support	Largest observation size; slightly more complex

Recommendation: Both **A** and **C** are excellent choices. As you noted (*"belki olabilir"*), **C (Multi-channel grid + inventory channel)** is mathematically richer and a very strong choice if you want to maximize the CNN's spatial awareness. Option **A** is a solid standard starting point.

1.3 Piece Orientation Encoding

Option	Description	Pros	Cons
A) All 8 orientations	Every piece has actions for all 8 orientations	Uniform action space (320 actions); simple	Redundant actions waste exploration
B) Map redundant	Internally deduplicate combinations	No wasted action slots; cleaner	Interface still has 320 actions

Recommendation: Use **A** for the action interface but implement **B** internally — mask out redundant orientation indices so the agent never picks them.

1.4 Reward Function Designs

Reward Function 1: Sparse Completion Reward

- Successful piece placement: $+1$
- Board fully solved (all 10 pieces): $+100$ bonus
- Invalid action: -10, episode ends
- Dead-end: 0, episode ends

Pros: Simple. *Cons:* Sparse; slow convergence.

Reward Function 2: Dense Shaped Reward

- Successful piece placement: $+4$ (= cells covered)
- Complete row: $+5$ bonus
- Contiguous empty region: $+2$ bonus
- Board fully solved: $+50$ bonus
- Invalid action: -5, episode ends
- Dead-end: $-$ (remaining empty cells)

Pros: Rich gradient signal. *Cons:* May bias the agent toward suboptimal strategies.

Recommendation: Compare **Reward 1 (sparse)** vs. **Reward 2 (dense shaped)**.

1.5 DRL Algorithm Choice

Option	Description	Pros	Cons
A) DQN	Value-based; discrete action fit	Well-understood; simple	Slow convergence; no stochastic policy
B) PPO	Policy gradient with clipped surrogate	State-of-the-art; stable; naturally stochastic	More complex to implement
C) A2C	Synchronous actor-critic	Simpler than PPO	Less stable; higher variance

Recommendation: Use **PPO** as the primary algorithm because its stochastic policy naturally discovers multiple solutions easily.

1.6 Network Architecture

Option	Description	Pros	Cons
A) MLP only	Flatten observation $\rightarrow$ 2-3 hidden layers	Simple; fast	Ignores spatial structure
B) CNN + MLP head	2-3 conv layers $\rightarrow$ flatten $\rightarrow$ MLP	Captures spatial patterns; good for grid	Board is very small
C) CNN + params	CNN on board, MLP on inventory vector	Best info separation	Most complex; overkill

Recommendation: Start with **B (CNN + MLP head)** — it's a good default for grid-based environments.

1.7 Episode Termination Conditions

Condition	When	Notes
Success	All 10 pieces placed, board covered	Return large positive reward
Invalid action	Agent picks out-of-bounds or overlapping	Only relevant if NOT using action masking
Dead-end	No valid actions exist for current piece	Episode must end; shaped reward penalty
Max steps	Count exceeds a limit (e.g., 50)	Safety net; unlikely if using masking

1.8 Piece Queue: Visible vs. Hidden

Option	Description	Pros	Cons
A) Full queue visible	Agent sees current piece + all remaining order	Agent can plan ahead; richer state	Much larger state space
B) Hidden queue	Agent sees current piece + remaining counts only	Simpler state; matches spec	Cannot plan for specific orderings

Recommendation: Use **B** — the spec requires "information about remaining pieces" but doesn't require order. Counts are simpler and more generalizable.

1.9 Coordinate Convention

Option	Description
A) Row-major	y-axis = row (top/bot), x-axis = col (L/R). Action = (orient, col, row).
B) Cartesian	x = col (L/R), y = row (bot/top). Action = (orient, x, y).

Recommendation: Use **A (row-major)** — it's the NumPy convention.

2 Detailed Task List — Split for 2 Members

2.1 Ground Rules

- Each member builds their **own complete pipeline** (environment + agent + training + eval).
- This ensures both can work fully in parallel without blocking each other.
- Common interfaces and hyperparameters are agreed upon upfront.
- At the end, combine the best results and write the report together.

2.2 Shared Agreements (Discuss Before Starting)

[ ] Agree on state representation (Option A or C).

[ ] Agree on action space: 320 flat discrete actions (8 orientations x 8 x 5).

[ ] Agree on coordinate convention: row-major.

[ ] Agree on piece shapes (offsets for each orientation of I, O, L, Z, T).

[ ] Agree on random seeds to use: [42, 123, 456, 789, 1024].

[ ] Set up shared Git repo with folder structure.

2.3 Suggested Folder Structure

```
cs545-project/
|-- common/                 # Shared utilities (pieces, visualize)
|   |-- pieces.py           # Piece shapes and orientations
|   |-- visualize.py        # Board visualization utility
|-- member_A/               # Member A's full pipeline (sparse reward)
|   |-- environment.py
|   |-- agent.py
|   |-- network.py
|   |-- train.py
|   |-- evaluate.py
|   |-- configs/
|-- member_B/               # Member B's full pipeline (dense reward)
|   |-- environment.py
|   |-- agent.py
|   |-- network.py
|   |-- train.py
|   |-- evaluate.py
|   |-- configs/
|-- results/                # Combined results for report
|-- report/                 # LaTeX report
|-- presentation/           # Slides
```

2.4 Member A — Tasks

Phase 1: Core Environment (Days 1–3)

- [ ] Implement piece definitions (I, O, L, Z, T).
- [ ] Implement BrainBlockEnv with **Sparse Reward**.
- [ ] Write unit tests for environment.
- [ ] Implement board visualization.

Phase 2: Agent & Training (Days 4–7)

- [ ] Implement PPO agent from scratch in PyTorch.
- [ ] Implement training loop (rollouts, updates, logging).
- [ ] Implement configurable hyperparameters.

Phase 3: Experiments & Evaluation (Days 8–10)

- [ ] Train with 5 random seeds.
- [ ] Implement evaluation script (success rate, return, length).

- [ ] Generate learning curve plots.
- [ ] Collect 5 distinct solutions.

Phase 4: Report Sections (Days 11–12)

- [ ] Write: MDP formulation, sparse reward explanation, and algorithms.

2.5 Member B — Tasks

Phase 1: Core Environment (Days 1–3)

- [ ] Implement piece definitions (I, O, L, Z, T).
- [ ] Implement BrainBlockEnv with **Dense Shaped Reward**.
- [ ] Write unit tests for environment.

Phase 2: Agent & Training (Days 4–7)

- [ ] Implement PPO agent (or DQN) for fair comparison.
- [ ] Implement training loop.

Phase 3: Experiments & Evaluation (Days 8–10)

- [ ] Train with 5 random seeds.
- [ ] Generate learning curve plots.
- [ ] Collect 5 distinct solutions + qualitative analysis.

Phase 4: Report Sections (Days 11–12)

- [ ] Write: Dense reward logic, comparative failure analysis, conclusions.

2.6 Joint Tasks (Days 12–14)

- [ ] Merge results and create side-by-side comparative figures.
- [ ] Finalize the report PDF.
- [ ] Prepare the 10-minute presentation slides.
- [ ] Prepare live visual demo script.

3 Key Hyperparameters to Start With

Parameter	**Suggested Value**
Learning rate	3e-4
Discount (γ)	0.99
GAE λ	0.95
PPO clip ϵ	0.2
Entropy coefficient	0.01
Value loss coefficient	0.5
Max grad norm	0.5
Rollout steps	2048
Mini-batch size	64
PPO epochs per update	4
Total training episodes	500,000+
Network hidden dim	256
CNN channels	[32, 64]
CNN kernel size	3x3 with padding